

Progetto di Fondamenti di Computer Grafica

Introduzione

Il progetto consiste nello sviluppo di un semplice videogioco 2D in cui il giocatore esplora un dungeon e combatte nemici. La progressione consiste nella sconfitta di orde di nemici e nell'apertura di forzieri che rendono accessibili percorsi fino ad allora bloccati.

Tappa 1

La tappa 1 consiste principalmente nel setup del progetto. Perciò è stata creata la finestra di gioco e sono state definite due strutture fondamentali: il giocatore (Player) e la gestione dello stato (State). È stato caricato lo sprite del giocatore. Non può ancora muoversi e non è animato.

Tappa 2

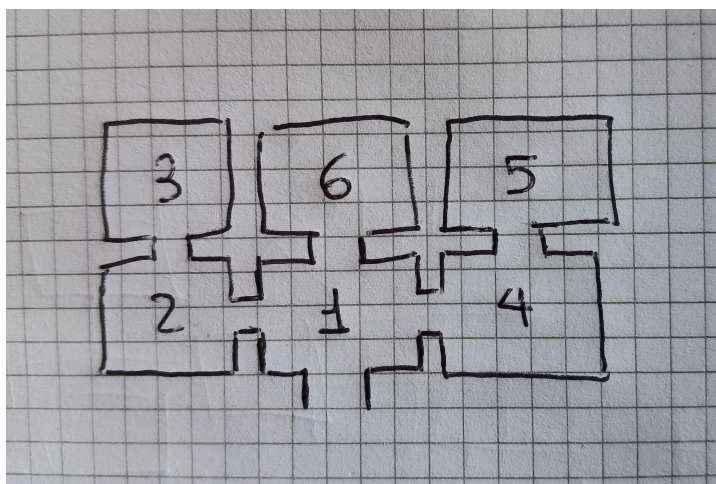
In questa fase, si è reso interattivo il personaggio principale. È stato implementato l'input di gioco per permettere il movimento del giocatore nello spazio. È stato inoltre integrato un sistema di animazioni basato sullo sprite sheet delle texture del giocatore. Ci sono animazioni di movimento e animazioni di riposo.

Tappa 3

In questa tappa è stato implementato il sistema di generazione delle stanze. La struttura delle stanze è definita in file JSON, per cui è stato sviluppato un parser in grado di generare la stanza leggendo gli elementi presenti nella griglia e traducendoli in tile di gioco (pavimenti e pareti). È stata creata una prima stanza con quattro aperture per i futuri collegamenti.

Tappa 4

In questa tappa il gioco si evolve da una singola stanza in una mappa di stanze connesse fra loro. Prima di tutto, è stata ideata la struttura del dungeon:



I numeri indicano il modo in cui è pensata la progressione, implementata più avanti. È stata poi aggiunta una hitbox al giocatore, collocata ai suoi piedi. La hitbox viene utilizzata nel sistema di transizione delle stanze: quando la hitbox supera alcune specifiche “linee” in corrispondenza delle aperture nella stanza, viene scaricata la stanza attuale e caricata la stanza che si collega a quella specifica apertura nella mappa del dungeon. La hitbox verrà anche utilizzata in seguito per le collisioni con l’ambiente e i nemici.

Tappa 5

A questo punto è stato aggiunto il sistema di collisione con le pareti. Quando la hitbox del giocatore interseca i tile corrispondenti alle pareti, la sua posizione viene immediatamente corretta, impedendo la compenetrazione nelle pareti. Sono inoltre stati aggiunti specifici oggetti all’interno delle stanze. Anche in questo caso, viene gestita la collisione del giocatore con questi oggetti, impedendo la compenetrazione con essi. Sono anche state aggiunte delle hitbox ai lati delle aperture delle grotte, per rendere più coerente con le texture la transizione tra le stanze.

Tappa 6

In questa tappa sono state aggiunte due tipologie di nemici con pattern di comportamento differenti. Il primo nemico è la Blue Slime. Questo nemico ha una “aggro zone”, ovvero una zona circolare attorno ad esso. Se il giocatore è presente all’interno di questa area, il nemico comincia ad attaccare, in questo caso balzando verso il giocatore. La aggro zone è stata implementata usando una struct personalizzata, `FloatCircle`, per calcolare la zona circolare attorno al nemico. Il secondo nemico è la Red Slime. Invece di balzare verso il giocatore, esso rimane fermo e sputa palle di fuoco verso il giocatore. La traiettoria di movimento della Blue Slime e delle palle di fuoco viene calcolata partendo dalla posizione del giocatore. Viene gestita la collisione delle Blue Slime con l’ambiente, per impedire che balzino dentro a una parete.

Tappa 7

Prima di cominciare questa tappa, l’unico file CPP utilizzato fino ad ora è stato suddiviso in più file per agevolare lo sviluppo dell’applicazione. Questa tappa introduce le meccaniche di attacco. L’attacco del giocatore consiste in un fendente di spada. L’attacco è animato e ha una propria hitbox. È stata aggiunta una “hurtbox” ai nemici, ovvero una zona rettangolare che ricopre il loro sprite. Se la hitbox del fendente e la hurtbox del nemico si intersecano, il nemico perde un punto vita. È stato inoltre aggiunto un effetto di lampeggiamento, gestito tramite fragment shader, allo sprite del nemico in caso di attacco andato a buon fine. Quando i punti vita del nemico vanno a 0, esso viene eliminato con un’animazione di morte.

Tappa 8

Da questa tappa in poi, il giocatore è vulnerabile agli attacchi nemici. Egli è dotato di punti vita e il contatto della propria hitbox con la hitbox dei nemici o delle palle di fuoco produce un danno. Dopo aver subito un danno, il giocatore è invincibile per 1 secondo. Lo sprite del

giocatore lampeggia rosso quando viene colpito ed è leggermente trasparente durante il periodo di invincibilità. Il giocatore può morire e ha una propria animazione di morte. Quando muore, appare una schermata di Game Over, e si può premere Enter per ricominciare la partita da capo.

Tappa 9

In questa tappa, il gioco si trasforma in un'avventura strutturata, con un inizio, un'evoluzione e una fine. Vengono introdotti dei flag di stato per gestire la progressione del gioco.

La progressione avviene così: le parti a destra e sopra la Stanza 1 sono inizialmente bloccate. La sconfitta di un'orda di nemici e l'apertura di un forziere nella Stanza 3 sblocca la parte a destra della Stanza 1. La Stanza 5 è uguale alla Stanza 3, si sconfiggono nemici e si apre un forziere che sblocca la Stanza 6. Aperto il forziere nella Stanza 6, il gioco finisce con una schermata di vittoria.

Il parser JSON e le mappe JSON sono state aggiornate per supportare stanze dinamiche che cambiano in base ai flag attivi. Sono stati aggiunti elementi scenici, come l'animazione di spawn per le orde di nemici, l'animazione delle spine e una meccanica di screen shake una volta aperto un forziere. Gli scrigni sono interagibili, quando si interagisce con essi viene attivata una textbox in basso con dei dialoghi.

Tappa 10

In questa tappa sono stati aggiunti elementi per rafforzare l'esperienza utente. È stato sviluppato un menù iniziale, in cui è possibile scegliere se giocare in modalità normale o in modalità difficile. Mentre si è in partita, è presente in alto a sinistra una sequenza di cuori che indica i punti vita rimanenti al giocatore. Come detto sopra, è stata implementata una modalità difficile il cui obiettivo è completare il gioco con un solo punto vita a disposizione. Inoltre, è stata creata una nuova stanza introduttiva (la Stanza 0), localizzata sotto la Stanza 1 nella mappa e con una sola transizione, verso la Stanza 1.

Tappa 11

Nell'ultima tappa, è stato implementato il comparto audio, sono stati rifiniti alcuni dettagli e aggiustati alcuni problemi.

Sono stati aggiunti effetti sonori per le azioni e musiche di sottofondo differenziate (per l'esplorazione del dungeon e per la schermata di vittoria).

È stato risolto un problema grafico in cui, ad esempio, il giocatore passa dietro a un nemico e il suo sprite appare "davanti" a quello del nemico invece che "dietro". I nemici e il giocatore ora vengono renderizzati in base alla coordinata Y della loro posizione, prima quelli "dietro" e poi quelli "davanti". È stato aggiustato un bug in cui la hurtbox dei nemici si allontanava dal loro sprite quando avveniva una collisione con una parete. Inoltre, il giocatore ora smette di muoversi quando incontra una parete.

Utilizzo dell'intelligenza artificiale

Durante lo sviluppo dell'applicazione, si è fatto ricorso all'utilizzo di applicazioni basate sull'IA generativa come Google Gemini per risolvere bug e per l'implementazione di funzionalità su cui si ha avuto difficoltà. Nei casi in cui l'IA ha generato codice, non è stato mai copiato pedissequamente, ma è stato rielaborato per fare in modo che si adatti alle specifiche esigenze del programma. L'IA è stata utilizzata in particolar modo per la gestione delle collisioni di nemici e del giocatore con le pareti e gli asset, dunque per i metodi `enemyCollisions` e `enemyWallCollisions` di `Room` e `processMovement` di `State`. È stata inoltre utilizzata considerevolmente per l'implementazione del `depth sorting`, dunque la parte dedicata a `render_queue` nel metodo `draw` di `State`, e per l'implementazione del costruttore di `ConditionEvaluator`.

Risorse

Il font utilizzato per le scritte è Public Pixel di GGBotNet.

Link: <https://www.dafont.com/public-pixel.font>

Tutti gli effetti sonori provengono dal videogioco *Deltarune* di Toby Fox.

La tracce musicali utilizzate per l'esplorazione del dungeon e per la schermata di vittoria sono rispettivamente "SWORD" e "Hall of Fame", entrambe di Toby Fox.

I file `assets-all.png` e `tiles-all-32x32.png` provengono da Basic Caves & Dungeons Tileset 32x32 Pixels di schwarnhild.

Link: <https://schwarnhild.itch.io/basic-caves-and-dungeons-tileset-32x32-pixels>

Il file `spikes.png` è una modifica fatta da me del file `assets-all.png`.

Il file `basic-tiles.png` proviene da Grass And Road Tiles Pixel Art di Mando37.

Link: <https://mando370.itch.io/top-down-grass-and-path-pixel-art>

Il file `HealthUI.png` proviene da Hearts UI (6 Lives) - Free Game Asset di SwooshWhoosh ed è stato modificato da me.

Link: <https://swooshwhoosh.itch.io/heartsui>

I file `player.png` e `slime.png` provengono da Mystic Woods di Game Endeavor.

Link: <https://game-endeavor.itch.io/mystic-woods>

Il file `redslime.png` è una modifica fatta da me del file `slime.png`.

Il file `smoke.png` proviene da Free FX - Pixel Art Effect FX062 di NYKNCK.

Link: <https://nyknck.itch.io/fx062?download>